

AURA-X

Autonomous Unified Reliability & Evolution Analyzer

GitHub Repository Integration & Ingestion System

Phase-Wise Implementation Plan

GitHub URL → Validation → GitHub API → Metadata → Branches → Commits → Secure Clone → Isolated Workspace → RepositoryContext → Code Intelligence → Evolution Analysis → Risk Prediction → Testing Pipeline → Database → REST API → Dashboard → Excel Reporting

Document: docs/GITHUB_INTEGRATION_PLAN.pdf

Owner: AURA-X Engineering (Backend / Git Integration / Security / API / QA)

Status: Draft for implementation kickoff

Guiding Principles

- The GitHub integration is the primary entry point into the AURA-X intelligence pipeline — not a bolt-on feature.
- Use REAL GitHub data only. No mock metadata, fake branches, hardcoded commits, or placeholder repository information.
- Downstream modules (Evolution Analysis, Risk Engine, Test Planning, Reporting) must depend only on the normalized **RepositoryContext** domain model, never on GitHub-specific classes directly.
- Clone and Execute are strictly separate phases. Cloning only retrieves source code; nothing is auto-executed.
- Secrets (GITHUB_TOKEN) are backend-only: never logged, never returned to clients, never embedded in clone URLs or error messages, never written to Excel.
- Every phase ends with tests passing and a documentation update before the next phase starts. No phase proceeds while a critical regression exists.

Architecture at a Glance

RepositoryProvider (abstract) → GitHubProvider (primary), LocalRepositoryProvider, GitLabProvider (future). All providers normalize into a single **RepositoryContext**: repository_id, provider, source_url, owner, repository_name, local_path, selected_branch, default_branch, commit_sha, metadata, branches, languages, file_tree, git_history, analysis_status, created_at, updated_at.

Ingestion pipeline: URL Validation → GitHub API Validation → Repository Metadata → Isolated Workspace Creation → Git Clone → Branch Checkout → Commit SHA Recording → Repository State Verification → RepositoryContext Creation → Persistence → API Exposure → Dashboard → Downstream Analysis Stages.

Phase Overview

#	Phase	Primary Outcome
1	Audit Existing Project	docs/github_integration_audit.md; reuse map; baseline tests green
2	Integration Architecture Design	RepositoryProvider + RepositoryContext design finalized
3	RepositoryProvider Abstraction	Provider interface + GitHubProvider skeleton wired into app
4	GitHub URL Parsing & Validation	Robust URL normalizer with structured error codes
5	GitHub API Client	Single client: metadata, branches, commits, languages, retries/rate-limits
6	Metadata & Branch Retrieval	Real repository profile + branch list + default-branch resolution
7	Secure Repository Cloning	Isolated, sandboxed, timeout-bound git clone + checkout + SHA capture
8	RepositoryContext Construction	Normalized context object with file inventory, languages, test frameworks
9	Database Persistence	Repository/Branch/Commit/AnalysisRun tables with valid state machine
10	REST API Layer	POST/GET endpoints per spec, Pydantic models, structured errors
11	Async Status Tracking	Real multi-stage ingestion status, pollable by frontend
12	Repository Intelligence & Evolution Hook	RepositoryContext feeds Evolution/Risk/Test Planning
13	Dashboard Integration	Real onboarding UI: paste URL, pick branch, watch real progress
14	Excel Reporting Integration	Repository facts flow into existing Excel pipeline, no secrets
15	Comprehensive Test Suite	Unit, API-client, git-integration, API, security, perf tests
16	End-to-End Validation	Real public repo flows fully through the pipeline

PHASE 1 | Audit Existing Project

Goal

Understand what already exists before writing any code, so nothing reusable is rewritten and nothing existing is broken.

Key Tasks

- Inspect backend architecture, frontend architecture, database models, existing API routes.
- Identify any existing Git or GitHub integration, repository analysis code, workflow orchestration.
- Identify existing tests, configuration, secret management, and domain models.
- Run the full baseline test suite and record current pass/fail status before any modification.

Deliverables

- docs/github_integration_audit.md documenting: existing implementation, reusable components, missing functionality, architectural integration points, risks, migration requirements.

Validation / Tests

- Baseline test suite executed and results captured (even if project is currently empty/new).

Exit Criteria (must pass before next phase)

Audit document exists and is reviewed; baseline tests (if any) are green or their failures are documented and accepted.

PHASE 2 | Design Integration Architecture

Goal

Define the RepositoryProvider abstraction and RepositoryContext domain model so the rest of AURA-X never talks to GitHub directly.

Key Tasks

- Design RepositoryProvider interface (fetch_metadata, list_branches, get_commit_history, clone, detect_languages).
- Design concrete providers: GitHubProvider now; LocalRepositoryProvider and GitLabProvider as future stubs.
- Define the normalized RepositoryContext schema (repository_id, provider, source_url, owner, repository_name, local_path, selected_branch, default_branch, commit_sha, metadata, branches, languages, file_tree, git_history, analysis_status, created_at, updated_at).
- Define the ingestion state machine: PENDING → VALIDATING → FETCHING_METADATA → FETCHING_BRANCHES → CLONING → SCANNING → READY / FAILED.
- Map how Repository Intelligence, Evolution Analysis, Dependency Analysis, Risk Engine, Test Planning, Test Generation/Execution, Failure Investigation, Repair, Verification, Learning Memory, and Reporting consume RepositoryContext.

Deliverables

- Architecture section added to docs/GITHUB_INTEGRATION.md with diagrams and interface signatures.
- Agreed RepositoryContext schema (versioned) shared with all downstream module owners.

Validation / Tests

- Design review checklist: no downstream module references GitHub types directly.

Exit Criteria (must pass before next phase)

RepositoryProvider interface and RepositoryContext schema are finalized and documented; no code depends on GitHub-specific types outside the provider layer.

PHASE 3 | Implement RepositoryProvider Abstraction

Goal

Stand up the abstract provider layer and register GitHubProvider as the active implementation.

Key Tasks

- Implement abstract base class/interface for RepositoryProvider.
- Implement provider registry/factory selecting a provider by source_url host (github.com now; extensible for gitlab.com etc.).
- Implement RepositoryContext as a concrete, serializable domain object independent of any provider SDK.
- Wire dependency injection so services request a RepositoryProvider rather than importing GitHub client code directly.

Deliverables

- app/domain (or equivalent) module: RepositoryProvider, RepositoryContext, provider factory.

Validation / Tests

- Unit tests for provider factory selection logic (unknown host raises UNSUPPORTED_REPOSITORY_PROVIDER).
- Unit tests confirming RepositoryContext round-trips through serialization.

Exit Criteria (must pass before next phase)

Provider abstraction compiles/imports cleanly and is unit-tested in isolation from any real GitHub network call.

PHASE 4 | GitHub URL Parsing and Validation

Goal

Robustly and safely turn arbitrary user input into a validated (owner, repository) pair, rejecting malicious or malformed input before any network or filesystem action occurs.

Key Tasks

- Implement a URL normalizer supporting https://github.com/owner/repo, .../repo.git, .../repo/ forms.
- Validate scheme, hostname, owner, repository name, path structure, character set.
- Reject unsupported hosts, path traversal sequences, and malformed paths with structured errors: INVALID_REPOSITORY_URL, UNSUPPORTED_REPOSITORY_PROVIDER, REPOSITORY_NOT_FOUND, REPOSITORY_ACCESS_DENIED.
- Extract and return normalized owner/repository strings for downstream use.

Deliverables

- github/url_parser module with parse_github_url(url) -> (owner, repo) and typed exceptions.

Validation / Tests

- Unit tests: valid URLs (with/without .git, with/without trailing slash).
- Unit tests: invalid scheme, wrong host, missing repo, path traversal (../, %2e%2e), null bytes, overlong input, unicode homograph attempts.

Exit Criteria (must pass before next phase)

100% of URL validation unit tests pass, including the full malicious-input security matrix.

PHASE 5 | Implement GitHub API Client

Goal

Centralize every GitHub HTTP call behind one client so no HTTP logic is scattered through the codebase.

Key Tasks

- Implement client methods: `get_repository`, `list_branches`, `get_branch`, `list_commits`, `get_commit`, `get_languages`, and optional `list_pull_requests` / `list_issues`.
- Implement timeouts, bounded retries with backoff, and pagination handling.
- Implement rate-limit detection (X-RateLimit-* headers) with structured `RATE_LIMIT_EXCEEDED` error and retry-after guidance.
- Implement optional Authorization header injection from `GITHUB_TOKEN` when configured; never log the token.
- Wrap all raw GitHub/HTTP exceptions into AURA-X structured error types before they leave the client.

Deliverables

- `github/api_client` module (single source of truth for GitHub HTTP access).
- Structured error taxonomy: `NOT_FOUND`, `ACCESS_DENIED`, `RATE_LIMITED`, `TIMEOUT`, `UPSTREAM_UNAVAILABLE`, `MALFORMED_RESPONSE`.

Validation / Tests

- Mocked-response tests: 200 success, 404, 401, 403, 429 rate limit, network timeout, malformed JSON, multi-page pagination.
- Verify no token value ever appears in logs, exceptions, or serialized error payloads.

Exit Criteria (must pass before next phase)

API client passes all mocked-response tests; zero direct GitHub HTTP calls exist anywhere else in the codebase.

PHASE 6 | Metadata and Branch Retrieval

Goal

Use the API client to build a real repository profile: identity, description, visibility, languages, and the full branch list with default-branch resolution.

Key Tasks

- Fetch repository metadata (id, name, owner, description, `default_branch`, visibility, topics, license, stars, forks, `open_issues`, `created_at`, `updated_at`) via `GitHubProvider`.
- Fetch branch list; resolve default branch when the caller omits one; validate a caller-supplied branch actually exists.

- Fetch language distribution via the languages endpoint.
- Fetch bounded, paginated commit history for the selected branch (author, date, message, sha, parents, changed files, additions/deletions), with a configurable history limit.

Deliverables

- GitHubProvider.fetch_metadata / list_branches / get_commit_history fully implemented against the real API.

Validation / Tests

- Integration-style tests against a small real public repository (network-gated, skippable offline).
- Unit tests for default-branch fallback and invalid-branch rejection.

Exit Criteria (must pass before next phase)

A real public repository URL returns accurate metadata, branch list, and commit history end-to-end.

PHASE 7 | Secure Repository Cloning

Goal

Clone actual source code into an isolated, sandboxed workspace — safely, with no shell injection and no automatic code execution.

Key Tasks

- Create an isolated workspace directory per repository_id (e.g. workspace/repositories/{id}/source), outside any publicly served path.
- Clone using subprocess with argument lists (never shell string interpolation); enforce clone timeout and repository size limits.
- Checkout the selected branch (or default branch) and record the resulting HEAD commit SHA.
- Verify post-clone state: .git exists, branch is checked out, HEAD resolves, repo is non-empty.
- Implement safe cleanup/eviction of stale workspaces; guard against path traversal in generated paths.
- Explicitly do NOT run install scripts, setup.py, package manager postinstall hooks, or any repository code.

Deliverables

- github/clone_service module: clone_repository(context) -> local_path, commit_sha, clone_status.

Validation / Tests

- Clone-success test against small temp local repo; invalid-URL clone failure; branch checkout test; empty-repository handling; cleanup-after-failure test.
- Security tests: shell-injection payloads in owner/repo/branch fields; oversized-repo rejection; clone-timeout enforcement.

Exit Criteria (must pass before next phase)

Clone pipeline is proven safe against injection/size/timeout abuse and produces a verifiable commit SHA for a real repository.

PHASE 8 | Construct RepositoryContext

Goal

Turn API metadata + cloned working tree into the single normalized object the rest of AURA-X consumes.

Key Tasks

- Scan the cloned tree: build file inventory (relative path, extension, size, category), respecting .gitignore and excluding .git, node_modules, venvs, caches, oversized/binary files.
- Detect languages via extension + GitHub language data + dependency/build files; primary target Python, future-ready for JS/TS/Java/C/C++.
- Detect test frameworks (pytest/unittest/tox/nox, Jest/Vitest/Mocha) and test directories without executing any commands.
- Compute Git evolution signals from commit history: file churn, modification frequency, recently changed files, additions/deletions, co-changing files, change concentration — feeding the Risk Engine.
- Assemble the final RepositoryContext and mark analysis_status = READY (or FAILED with structured error).

Deliverables

- RepositoryContext fully populated for a real repository, including file_tree, languages, git_history, and evolution signals.
- Repository Profile view (identity, metadata, branch, commit SHA, languages, file inventory, test frameworks, dependencies, git history summary).

Validation / Tests

- Unit tests for file discovery/exclusion rules, language detection, test-framework detection.
- Unit tests for churn/co-change calculations against a fixture commit history.

Exit Criteria (must pass before next phase)

RepositoryContext for a real public repo contains accurate, non-empty file inventory, languages, test-framework detections, and evolution signals.

PHASE 9 | Database Persistence

Goal

Persist repository, branch, commit, and analysis-run state using the existing database architecture — no parallel storage system.

Key Tasks

- Model Repository, Branch, Commit reference, AnalysisRun, and ingestion status/state-machine tables (reusing existing DB/ORM conventions from the audit in Phase 1).
- Implement valid state transitions: PENDING → VALIDATING → FETCHING_METADATA → FETCHING_BRANCHES → CLONING → SCANNING → READY / FAILED, rejecting invalid jumps.
- Persist structured errors against the AnalysisRun so failures are diagnosable after the fact.
- Guarantee reproducibility: every AnalysisRun records exactly which repository, branch, commit SHA, and configuration were analyzed.

Deliverables

- DB migrations/models for Repository, Branch, AnalysisRun, IngestionStatus.
- Repository (data-access) layer used by services instead of ad hoc queries.

Validation / Tests

- State-machine unit tests (valid transitions accepted, invalid transitions rejected).

- Persistence round-trip tests (write then read back an AnalysisRun with full lineage).

Exit Criteria (must pass before next phase)

A completed ingestion is fully persisted and re-readable, with an auditable repository → branch → commit → analysis-run chain.

PHASE 10 | REST API Layer

Goal

Expose ingestion, browsing, and profile endpoints using the project's existing API conventions.

Key Tasks

- POST /api/v1/repositories/github — accepts {repository_url, branch?}, returns repository_id, provider, source_url, name, owner, selected_branch, commit_sha, status.
- GET /api/v1/repositories, GET /api/v1/repositories/{id}, GET /api/v1/repositories/{id}/branches, GET /api/v1/repositories/{id}/profile, GET /api/v1/repositories/{id}/commits.
- POST /api/v1/repositories/{id}/refresh to re-ingest.
- Pydantic request/response models; structured error responses (no raw GitHub exceptions leak through); pagination on list endpoints; auth enforced per existing app conventions.

Deliverables

- Fully documented (OpenAPI) endpoint set matching the spec above.

Validation / Tests

- API tests: valid public repo ingestion, invalid URL, branch selection, profile retrieval, refresh, status polling, pagination, auth-required paths.

Exit Criteria (must pass before next phase)

All listed endpoints exist, are documented in OpenAPI, and pass their API test suite.

PHASE 11 | Asynchronous Status Tracking

Goal

Ensure long-running ingestion never blocks the frontend, and that reported progress reflects real stages, not a simulated progress bar.

Key Tasks

- Implement background job execution for ingestion (queue/worker or async task per existing app architecture from the audit).
- Expose real status values: QUEUED, VALIDATING, FETCHING, CLONING, ANALYZING, READY, FAILED, each set only when that stage genuinely starts/completes.
- Provide a status-polling endpoint (or reuse GET /repositories/{id}) reflecting live state.

Deliverables

- Background ingestion worker/task wired to the state machine from Phase 9.

Validation / Tests

- Test that status transitions occur in the correct order and only after the corresponding real work completes.
- Test failure mid-pipeline surfaces FAILED with the structured error, not a silent hang.

Exit Criteria (must pass before next phase)

Frontend can poll real, monotonically progressing status for an in-flight ingestion of a real repository.

PHASE 12 | Connect Repository Intelligence, Evolution Analysis, Risk & Test Planning

Goal

Feed the completed RepositoryContext into the existing (or newly stubbed) downstream AURA-X stages so the pipeline is genuinely end-to-end.

Key Tasks

- Pass RepositoryContext into Repository Intelligence, Evolution Analysis, Dependency Analysis, Risk Assessment, and Test Planning without requiring redundant manual input.
- Ensure repository/branch/commit selection stays consistent across every downstream stage for a single run.
- Verify Evolution Analysis actually consumes the churn/co-change signals computed in Phase 8.

Deliverables

- Downstream module entry points accepting RepositoryContext as their sole repository input.

Validation / Tests

- Integration test: one RepositoryContext flows unchanged (same commit SHA) through Intelligence → Evolution → Risk → Test Planning.

Exit Criteria (must pass before next phase)

A single ingestion run produces one RepositoryContext that downstream stages consume without re-deriving repository facts themselves.

PHASE 13 | Dashboard Integration

Goal

Give the user a real onboarding UI backed entirely by the APIs built in Phase 10-11.

Key Tasks

- Build the repository onboarding screen: paste GitHub URL, show validation errors, show fetched repository info and branches, allow branch selection, start analysis.
- Show real ingestion progress driven by the status endpoint (no simulated/fake progress bar).
- Render the resulting Repository Profile: name, owner, description, selected branch, commit SHA, languages, statistics, detected test frameworks, ingestion status.

Deliverables

- Repository onboarding page/component wired to real REST endpoints, no hardcoded repository cards.

Validation / Tests

- Manual/E2E browser walkthrough: paste a real public GitHub URL, select a branch, observe real progress and a populated profile.

Exit Criteria (must pass before next phase)

A user can complete the full onboarding flow in the browser using only real API data.

PHASE 14 | Excel Reporting Integration

Goal

Route repository facts into the existing Excel export pipeline using the same domain models as the API and dashboard — not a separate data path.

Key Tasks

- Add repository URL, provider, owner, repository name, selected branch, commit SHA, primary language, language distribution, and analysis timestamp to the relevant Excel sheet(s).
- Source this data from the same Repository/AnalysisRun models used by the API layer.
- Explicitly verify GITHUB_TOKEN or any credential never appears in generated workbooks.

Deliverables

- Updated Excel export including the repository identity/metadata block.

Validation / Tests

- Snapshot test asserting expected fields present and no secret-like values in generated Excel output.

Exit Criteria (must pass before next phase)

Excel export for a completed run includes accurate repository facts with zero credential leakage.

PHASE 15 | Comprehensive Test Suite

Goal

Reach full coverage across unit, API-client, git-integration, API, and security dimensions, without making the whole suite permanently dependent on internet access.

Key Tasks

- Unit tests: URL parsing/normalization, provider detection, owner/repo extraction, branch validation, RepositoryContext creation, state transitions, error mapping.
- API client tests: mocked success/404/401/403/429/timeout/malformed-response/pagination.
- Git integration tests: temporary local repos for clone success/failure, branch checkout, commit SHA, empty repo, cleanup.
- API tests: repository creation, invalid URL, valid public repo, branch selection, profile, refresh, status tracking.
- One clearly-marked, configurable integration test against a real small public repository (skippable offline).
- Security tests: shell injection, malformed URLs, unsupported protocols, malicious repo names, path traversal, oversized repos, token-leakage checks across logs/errors/responses/reports.

Deliverables

- Full automated test suite runnable in CI, offline-safe by default, with an opt-in network-integration tier.

Validation / Tests

- CI run: full suite green offline; network-tier run green when enabled.

Exit Criteria (must pass before next phase)

Entire test suite (unit + client + git + API + security) passes; no credentials appear anywhere in test output or fixtures.

PHASE 16 | End-to-End Validation

Goal

Prove the complete pipeline works for a real user with a real public repository, exactly as specified in the completion requirements.

Key Tasks

- Walk the full flow: paste URL → validate → fetch metadata/branches → select branch → clone → record commit SHA → scan → detect languages/test frameworks → build RepositoryContext → persist → expose via API → render on dashboard → hand off to downstream analysis.
- Confirm reproducibility: re-running the same URL/branch either reuses valid cached analysis for that exact commit SHA or produces a fresh, consistent run.
- Confirm no mock data, hardcoded values, or fake progress exist anywhere in the shipped path.
- Finalize docs/GITHUB_INTEGRATION.md and README updates (env vars, GITHUB_TOKEN setup, how to analyze a repository).

Deliverables

- Signed-off end-to-end demo using a real public repository (e.g. a small well-known open-source repo).
- Finalized docs/GITHUB_INTEGRATION.md and README section.

Validation / Tests

- Full manual/E2E regression pass across API, dashboard, and Excel export for the demo repository.

Exit Criteria (must pass before next phase)

A real public GitHub repository genuinely enters and completes the AURA-X pipeline end-to-end with no mocked step.

Cross-Cutting Requirements (apply to every phase)

- **Security:** no shell interpolation, argument-list subprocess calls only; strict URL/path validation; never execute cloned repository code during ingestion.
- **Secrets:** GITHUB_TOKEN read from environment/approved secret storage only, backend-only, never logged or returned to clients, never in clone URLs that get logged, never in Excel.
- **Errors:** every failure mode (invalid URL, not found, private/access denied, invalid branch, GitHub unavailable, rate limit, timeout, clone failure, oversized repo, empty repo, disk/workspace failure, invalid git repo, commit not found, git not installed) maps to a structured, actionable, safely-logged error.
- **Reproducibility:** every analysis run is pinned to an exact repository + branch + commit SHA + configuration.

- **Performance:** bounded commit history, no redundant re-cloning or re-fetching for an unchanged repository/branch/commit; caching keyed strictly by (repository, branch, commit_sha).
- **Process:** after every phase — run relevant tests, fix failures immediately, verify integration with prior phases, verify real data flow end-to-end for that phase, update documentation, and do not proceed while a critical regression exists.

This plan operationalizes the full GitHub Integration specification into 16 sequential, independently testable phases. Each phase produces a real, working increment of the pipeline — never a placeholder — so that at every checkpoint a real public GitHub repository can be pushed further down the AURA-X pipeline than it could before.